

Guide de soutenance — Predicting User Sentiment in Software Issue Reports

ST2MLE · Tout ce qu'il faut savoir pour défendre le projet

But de ce document : te permettre de connaître le projet sur le bout des doigts — le rôle de chaque fichier, le déroulé complet, **chaque métrique expliquée dans notre contexte précis**, et un **Q&A anticipé** pour le jury.

0. Le pitch de 60 secondes (à savoir réciter)

« On classe le **sentiment** d'avis d'applications (avis Facebook réels, Kaggle) en **négatif / neutre / positif**. On part du **texte brut**, on le **nettoie** (minuscules, ponctuation, lemmatisation), on le transforme en **vecteurs** (Bag-of-Words, TF-IDF, Word2Vec entraîné sur nos données **et** Word2Vec pré-entraîné Google-News), puis on **compare 5 classifieurs** (Naive Bayes, arbre de décision, forêt aléatoire, AdaBoost, régression logistique), chacun **réglé par validation croisée 5-fold + GridSearchCV**. On évalue avec **accuracy, précision, rappel, F1 et matrice de confusion**. Le meilleur est **TF-IDF + Naive Bayes (macro-F1 = 0.559)**. Le point clé : la classe **neutre** ne représente que 4.5 % des données, donc **l'accuracy est trompeuse** (0.84 alors que le modèle ne prédit jamais "neutre") — c'est le **macro-F1** qui révèle le problème, et **SMOTE** qui le corrige. »

Les 5 chiffres à retenir absolument :

Chiffre	Signification
5 923	avis uniques (après déduplication des 10 000 bruts)
64 % / 31 % / 4.5 %	positif / négatif / neutre (déséquilibre fort)
0.559	macro-F1 du meilleur modèle (TF-IDF + Naive Bayes)
0.839	accuracy du même modèle (>> macro-F1 → piège du déséquilibre)
0.000 → 0.472	rappel de la classe neutre : baseline → après sous-échantillonnage

1. Vue d'ensemble

- **Problème** : classification de texte **multi-classe** (3 classes de sentiment).
- **Tâche d'apprentissage** : **supervisée** (on connaît le label de chaque avis).
- **Entrée** : le texte d'un avis. **Sortie** : une classe parmi `negative`, `neutral`, `positive`.
- **Pourquoi c'est utile** : trier automatiquement les retours utilisateurs, repérer les avis négatifs à traiter en priorité, mesurer la satisfaction.
- **Ce qu'on compare** : 4 façons de représenter le texte × 5 algorithmes de classification = **20 combinaisons**, plus 4 expériences dédiées (déséquilibre, réduction de dimension, élagage, early stopping).

2. Le dataset (à connaître précisément)

- **Source** : export Kaggle d'**avis d'applications Facebook** (`content` = texte, `score` = note 1 à 5 étoiles). 10 000 avis bruts.
- **Création des labels** (étape clé à justifier) : on **dérive le sentiment de la note** (convention standard) :
- **1-2 ★ → négatif, 3 ★ → neutre, 4-5 ★ → positif.**
- **Déduplication** : beaucoup d'avis sont identiques ("good", "nice", "love it"). On supprime les doublons de texte → **5 923 avis uniques**.
- Pourquoi ? Si le même texte est à la fois en train et en test, le modèle "triche" (fuite de données / data leakage). La déduplication garantit une évaluation honnête.
- **Distribution finale** (déséquilibrée, naturelle) :

Classe	Note	Nombre	Part
positive	4-5 ★	3 801	64.2 %
negative	1-2 ★	1 857	31.4 %
neutral	3 ★	265	4.5 %

- **Pourquoi le neutre est rare et difficile** : un avis 3★ est souvent "ça marche mais bof" — un label **proxy** (approximatif), tonalement ambigu, qui chevauche les deux autres classes.

Phrase de défense : « Les labels viennent des étoiles, c'est un signal proxy : pratique et standard, mais imparfait — c'est l'une des raisons pour lesquelles la classe neutre est si dure. »

3. Architecture du code — qui fait quoi

```
config.py          ← Paramètres centraux (graine 42, chemins, tailles, grilles)
main.py            ← Chef d'orchestre : lance tout le pipeline de bout en bout
data/
  facebook_reviews.csv ← Données brutes (avis + notes)
  issue_sentiment.csv  ← Données traitées et mises en cache (texte + label)
src/
  real_data.py         ← Charge les avis, mappe étoiles→sentiment, échantillonne
  data_generation.py   ← Générateur synthétique de secours + aiguilleur de source
  preprocessing.py    ← Nettoyage du texte (minuscule, ponctuation, lemmatisation)
  vectorization.py     ← BoW, TF-IDF, Word2Vec, Word2Vec pré-entraîné, BERT
  balancing.py         ← SMOTE et sous-échantillonnage (imbalanced-learn)
  models.py           ← Les 5 classifieurs + leurs grilles d'hyperparamètres
  evaluation.py        ← Métriques (accuracy/precision/recall/F1) + graphiques
  experiment.py        ← Assemble tout : entraînement, CV, 4 expériences
scripts/
  build_report_pdf.py / build_slides_pdf.py ← génèrent les PDF
results/           ← Sorties : tableaux CSV, matrices de confusion, figures
report/            ← REPORT.md/pdf, SLIDES.md/pdf, ce guide
```

Le flux de données (à dessiner au tableau si besoin) :

```

texte brut
→ preprocessing.py      (nettoyage : minuscule, ponctuation, stop-words, lemmes)
→ vectorization.py      (texte → vecteurs de nombres)
→ [balancing.py]        (SMOTE / sous-échantillonnage – sur le train uniquement)
→ [TruncatedSVD]        (réduction de dimension – optionnelle)
→ models.py             (classifieur, réglé par GridSearchCV + CV 5-fold)
→ evaluation.py          (accuracy, précision, rappel, F1, matrice de confusion)

```

4. Le pipeline étape par étape (le QUOI + le POURQUOI)

4.1 Prétraitement (preprocessing.py)

Suite classique : **minuscules** → **suppression URLs/HTML/ponctuation/chiffres** → **tokenisation** → **suppression des stop-words** → **lemmatisation (WordNet)**.

Deux choix "intelligents" à savoir justifier : - **On garde les négations** (not , no , never ...). Pourquoi ? "not good" et "good" ont des sentiments opposés ; supprimer "not" comme un stop-word ordinaire détruirait le signal. - **Lemmatisation en deux passes** (verbe puis nom) : "crashing" → "crash", "issues" → "issue". Pourquoi ? Regrouper les formes d'un même mot réduit le vocabulaire sans avoir besoin d'un analyseur grammatical coûteux. -

Exemple : "Critical issue -- The app keeps crashing!!!" → critical issue app keep crash.

Stop-words = mots très fréquents et peu informatifs ("the", "a", "is...") qu'on retire pour réduire le bruit. **Lemmatisation** = ramener un mot à sa forme de base (le lemme).

4.2 Vectorisation (vectorization.py) — texte → nombres

Les modèles ne comprennent que des nombres. On transforme chaque avis en vecteur.

Méthode	Idée	Sortie
Bag-of-Words (BoW)	compte combien de fois chaque mot apparaît	creux, ≥ 0
TF-IDF	comme BoW, mais pondère : un mot rare et discriminant pèse plus qu'un mot omniprésent	creux, ≥ 0
Word2Vec	chaque mot → vecteur dense appris sur nos 5 923 avis ; on moyenne les mots de l'avis	dense
Word2Vec-pretrained	idem mais vecteurs pré-entraînés Google News (300-d, 3 M mots) — c'est le « pre-trained Word2Vec » exact du sujet	dense
BERT	embeddings contextuels (code fourni, optionnel)	dense

- **TF-IDF** = Term Frequency × Inverse Document Frequency. Un mot qui apparaît partout (peu discriminant) est dévalué ; un mot rare et spécifique est valorisé.
- **"Creux" (sparse)** = la plupart des valeurs sont 0 (un avis n'utilise qu'une poignée des milliers de mots du vocabulaire). **"Dense"** = toutes les valeurs sont non nulles.
- **n-grammes (1,2)** : on prend les mots seuls et les paires ("not good"), ce qui capture un peu de contexte.

- **Deux Word2Vec ?** Oui, et c'est volontaire : le sujet demande le « pre-trained Word2Vec » → c'est `Word2Vec-pretrained` (Google News). On garde en plus un Word2Vec **entraîné sur nos avis** pour comparer pré-entraîné vs appris en interne. Les deux sont l'algorithme Word2Vec (pas GloVe).

Word2Vec vs GloVe (si on te le demande) : ce sont deux méthodes d'embeddings de mots. Le sujet nomme **Word2Vec** → c'est ce qu'on utilise (Google News). On n'utilise **pas** GloVe pour rester strictement dans le périmètre du sujet.

Pourquoi 4 méthodes ? Sur des avis courts, les méthodes **par comptage (TF-IDF/BoW)** gagnent ; les embeddings moyennés "diluent" les mots-clés de sentiment.

4.3 Découpage train / test

On sépare **80 % entraînement / 20 % test**, en **stratifié** (les proportions des 3 classes sont conservées dans chaque partie). Le test n'est utilisé **qu'à la fin** pour mesurer la performance sur des données jamais vues.

4.4 Validation croisée (CV) + GridSearchCV (`experiment.py` , `models.py`)

- **Validation croisée 5-fold** : on découpe le train en 5 morceaux ; on entraîne sur 4, on valide sur 1, et on tourne 5 fois. La performance = moyenne des 5. Pourquoi ? Estimation **plus fiable** et moins dépendante d'un seul découpage.
- **GridSearchCV** : teste **toutes les combinaisons** d'hyperparamètres d'une grille, et garde la meilleure (selon le score de CV). C'est le **réglage (tuning)** des modèles.
- **Hyperparamètre** = réglage qu'on fixe avant l'entraînement (ex. profondeur d'un arbre), par opposition aux paramètres appris (les poids).

Anti-fuite de données : le réglage se fait **uniquement sur le train** (via la CV). Le test reste vierge → la performance annoncée est honnête.

4.5 Gestion du déséquilibre (`balancing.py`)

La classe `neutral` ne fait que 4.5 %. Deux remèdes comparés : - **SMOTE** (sur-échantillonnage) : crée de **nouveaux** exemples neutres synthétiques en interpolant entre voisins → enrichit la minorité sans copier. - **Sous-échantillonnage (under-sampling)** : **supprime** des exemples des classes majoritaires jusqu'à égaliser → simple mais jette de l'information. - **Crucial** : appliqué **uniquement sur le train** (via une `Pipeline imblearn`), jamais sur le test → pas de fuite.

4.6 Réduction de dimension — PCA / TruncatedSVD (`experiment.py`)

- On a ~4 600 colonnes (le vocabulaire). **TruncatedSVD** projette sur beaucoup moins de dimensions tout en gardant l'essentiel de l'information (la "variance").
- **Pourquoi SVD et pas PCA "classique" ?** La PCA centre les données, ce qui rendrait notre matrice creuse **dense** (coûteux en mémoire). TruncatedSVD est l'équivalent adapté au texte creux (aussi appelé **LSA**). On l'appelle "PCA" car le sujet le demande, mais on explique le choix.

- **Résultat** : 300 dimensions (6.5 % des features) gardent ~99 % de la performance → gros gain de vitesse/mémoire.

4.7 Élagage de l'arbre (pré et post) — `models.py` + `experiment.py`

Un arbre de décision non contrôlé **surapprend** (mémorise le train). - **Pré-élagage** : on limite la croissance pendant la construction (`max_depth` , `min_samples_leaf`). Réglé dans la grille. - **Post-élagage** : on laisse l'arbre pousser, puis on **coupe** les branches peu utiles (`ccp_alpha` , cost-complexity pruning). - **Résultat** : de **1 857 nœuds** → **69 nœuds**, l'accuracy test **monte de 0.76 à 0.79** (l'écart train-test passe de 0.22 à 0.03) → moins de surapprentissage.

4.8 Régularisation

La **régression logistique** pénalise les poids trop grands (**régularisation L2**) pour éviter le surapprentissage. On règle sa force via l'hyperparamètre `C` (petit `C` = forte régularisation). C'est notre démonstration de régularisation.

4.9 Early stopping (arrêt précoce)

Pour le **gradient boosting** (modèle qui ajoute des arbres un par un), on autorise jusqu'à 500 arbres mais on **arrête dès que la performance de validation stagne** (`n_iter_no_change`). **Résultat** : arrêt à **88 arbres sur 500** → ~6× moins de calcul, sans perte de qualité, et protection contre le surapprentissage.

5. Les 5 modèles (intuition + forces/faiblesses)

Modèle	Intuition en une phrase	Force	Faiblesse
Naive Bayes (<code>MultinomialNB</code>)	probabilités des mots par classe, en supposant les mots indépendants	très rapide, robuste en grande dimension, gagnant ici	l'hypothèse d'indépendance est fausse
Arbre de décision	suite de questions oui/non sur les mots	interprétable	surapprend → besoin d'élagage
Forêt aléatoire	moyenne de beaucoup d'arbres dépareillés (bagging)	robuste, peu de réglage	moins lisible, plus lourd
AdaBoost	combine des "souches" (arbres à 1 décision) en se focalisant sur les erreurs	bon sur features denses	mauvais ici : une souche ne voit qu'1 mot sur des milliers
Régression logistique	frontière linéaire pondérée, régularisée L2	solide baseline, calibrée	frontière linéaire seulement

MultinomialNB vs GaussianNB : MultinomialNB exige des valeurs **positives** (comptages) → parfait pour BoW/TF-IDF. Les embeddings (Word2Vec, pré-entraîné ou

non) ont des valeurs **négatives** → on bascule **automatiquement** sur GaussianNB.
(Bon point à mentionner : "on adapte le modèle à la nature des features".)

6. LES MÉTRIQUES — expliquées dans NOTRE contexte (section clé)

6.1 La brique de base : TP, FP, FN, TN (par classe)

En multi-classe, chaque métrique se calcule **classe par classe**, en mode "**une-contre-le-reste**" (one-vs-rest). Pour une classe donnée (ex. `neutral`) :

- **VP / TP (Vrai Positif)** : avis vraiment neutres, prédits neutres.
- **FP (Faux Positif)** : avis prédits neutres mais qui ne le sont pas.
- **FN (Faux Négatif)** : avis vraiment neutres mais ratés (prédits autre chose).
- **VN / TN (Vrai Négatif)** : avis non-neutres, prédits non-neutres.

6.2 Accuracy (exactitude)

$$\text{Accuracy} = \frac{\text{prédictions correctes}}{\text{total}}$$
 - **En français** : "sur tous les avis, quelle proportion est bien classée ?" - **Notre piège** : 64 % des avis sont positifs. Un modèle qui dirait toujours positif aurait déjà ~64 % d'accuracy **sans rien comprendre**. → l'accuracy **surévalue** la performance sur données déséquilibrées.

6.3 Precision (précision)

$$\text{Precision} = \frac{TP}{TP + FP}$$
 - **En français** : "**parmi les avis que le modèle a étiquetés classe X**, combien le sont vraiment ?" → mesure la **fiabilité des alertes**. - Ex. $\text{precision}(\text{négatif})$ élevée = "quand le modèle dit négatif, on peut lui faire confiance".

6.4 Recall (rappel / sensibilité)

$$\text{Recall} = \frac{TP}{TP + FN}$$
 - **En français** : "**parmi les avis qui sont vraiment de la classe X**, combien le modèle en retrouve ?" → mesure la **couverture**. - Ex. $\text{recall}(\text{neutre}) = 0.000$ → "le modèle n'a retrouvé AUCUN des avis neutres".

Precision vs Recall (à savoir distinguer absolument) : Précision = qualité de ce qu'on annonce (peu de fausses alertes). Rappel = quantité de ce qu'on attrape (peu d'oublis). Il y a souvent un **compromis** : forcer plus de neutres (↑ rappel) crée plus de fausses alertes (↓ précision).

6.5 F1-score

$$F_1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$
 - **Moyenne harmonique** de précision et rappel → un seul nombre qui résume les deux. Il est **bas si l'un des deux est bas** (contrairement à la moyenne simple).

6.6 Macro vs Weighted vs Micro

On a un F1 par classe ; comment résumer en un chiffre ? - **Macro-F1** = moyenne **simple** des 3 F1 : $\$(F1_{\text{nég}} + F1_{\text{neu}} + F1_{\text{pos}})/3\$$. → **traite les 3 classes à égalité** : rater la classe rare coûte cher. - **Weighted-F1** = moyenne **pondérée par le nombre d'avis** de chaque classe. → dominée par la classe majoritaire (positif). - **Micro-F1** = agrège tous les TP/FP/FN globalement → en multi-classe, égale l'accuracy.

Pourquoi on a choisi le macro-F1 comme métrique de réglage

(`SCORING="f1_macro"`) : c'est la seule qui **refuse de récompenser** un modèle qui ignore la classe neutre. C'est LA justification centrale du projet.

6.7 Exemple chiffré COMPLET sur notre meilleur modèle

Modèle **TF-IDF + Naive Bayes** (alpha = 0.5), sur les **1 185** avis de test. **Matrice de confusion** (lignes = vraie classe, colonnes = classe prédite) :

	préd. négatif	préd. neutre	préd. positif	Total réel
vrai négatif	291	0	81	372
vrai neutre	22	0	31	53
vrai positif	57	0	703	760
Total prédit	370	0	815	1185

Observation choc : la **colonne "neutre" est entièrement à 0** → le modèle **n'a jamais prédit "neutre"** une seule fois ! Les 53 vrais neutres ont été versés dans négatif (22) ou positif (31).

Calculs (à savoir refaire au tableau) : - **Accuracy** = diagonale / total = $(291 + 0 + 703) / 1185 = 994/1185 = 0.839$ - **Négatif** : précision = $291/370 = 0.786$; rappel = $291/372 = 0.782$; F1 = **0.784** - **Neutre** : précision = $0/0 \rightarrow$ définie à **0.000** ; rappel = $0/53 = 0.000$; F1 = **0.000** - **Positif** : précision = $703/815 = 0.863$; rappel = $703/760 = 0.925$; F1 = **0.893** - **Macro-F1** = $(0.784 + 0.000 + 0.893) / 3 = 0.559$ - **Weighted-F1** = $(372 \cdot 0.784 + 53 \cdot 0 + 760 \cdot 0.893) / 1185 = 0.819$

La phrase qui tue : « L'accuracy est de 0.84, le weighted-F1 de 0.82... mais le **macro-F1 n'est que 0.56**, parce que le modèle **ignore totalement la classe neutre** (F1 = 0). C'est exactement pourquoi, sur un problème déséquilibré, on regarde le **macro-F1** et le **rappel par classe**, pas l'accuracy. Et c'est ce que **SMOTE** corrige. »

7. La matrice de confusion — comment la lire

- **Définition** : tableau qui croise la **vraie classe** (lignes) et la **classe prédite** (colonnes).
- **La diagonale** = bonnes prédictions. **Hors diagonale** = erreurs (et on voit vers quelle classe le modèle se trompe).

- Chez nous, lire la matrice révèle instantanément le problème : colonne neutre vide.
Une simple accuracy ne l'aurait jamais montré.
- Fichiers : une matrice par combinaison dans `results/confusion_matrices/`.

8. Résultats & comment les justifier

Top du tableau de comparaison (test, trié par macro-F1) :

Rang	Vectoriseur	Modèle	macro-F1	accuracy
1	TF-IDF	Naive Bayes	0.559	0.839
2	TF-IDF	Régression logistique	0.555	0.830
3	TF-IDF	Forêt aléatoire	0.548	0.824
4	BoW	Naive Bayes	0.548	0.817
...	...	...	...	...
20	BoW	AdaBoost	0.465	0.753

Points à défendre : 1. **TF-IDF + Naive Bayes gagne** : sur du texte court, les comptages pondérés + NB (rapide, robuste en grande dimension) sont difficiles à battre. Le **top 5 est entièrement TF-IDF/BoW** → les méthodes par comptage dominent les embeddings. 2. **Scores serrés (0.46-0.56)** : tous les modèles butent sur la **même difficulté** — la classe neutre minuscule. 3. **AdaBoost est dernier (0.46)** : ses souches (1 mot testé à la fois) sont inadaptées à un vocabulaire de milliers de mots creux. 4. **Embeddings au milieu (0.49-0.54)** : pré-entraîné (Google News) comme in-domaine, la moyenne des vecteurs "dilue" les mots-clés de sentiment des avis courts. Le Word2Vec pré-entraîné fait mieux avec Random Forest (0.542) que le Word2Vec interne → "pré-entraîné" n'est pas automatiquement "meilleur".

Le résultat le plus marquant — l'expérience de déséquilibre :

Stratégie	accuracy	macro-F1	rappel neutre
aucune (baseline)	0.836	0.555	0.000
SMOTE	0.727	0.551	0.189
sous-échantillonnage	0.601	0.505	0.472

→ La baseline atteint 0.84 d'accuracy **en ne prédisant jamais "neutre"**. SMOTE et le sous-échantillonnage **forcent** le modèle à reconnaître la minorité (rappel neutre 0 → 0.19 → 0.47), au prix de l'accuracy. **C'est le compromis fondamental de l'apprentissage déséquilibré.**

9. Q&A anticipé — questions probables du jury

Q1. Pourquoi 3 classes et pas 2 (positif/négatif) ? Le sujet demande positif/neutre/négatif. Le neutre (3★) est le plus dur et rend le problème réaliste. (On pourrait faire du binaire en retirant les 3★ — piste future.)

Q2. Pourquoi le macro-F1 plutôt que l'accuracy ? Parce que 64 % des avis sont positifs : l'accuracy récompense un modèle qui ignore les classes rares. Le macro-F1 traite les 3 classes à égalité et révèle ce défaut.

Q3. Votre meilleur modèle a 0.84 d'accuracy, c'est bien, non ? Trompeur : il **ne prédit jamais "neutre"** (F1 neutre = 0). Le macro-F1 (0.56) et le rappel par classe le montrent. La vraie qualité est modeste — c'est honnête sur des données réelles bruitées.

Q4. Différence précision / rappel ? Précision = "quand je dis X, ai-je raison ?" (fiabilité). Rappel = "ai-je trouvé tous les X ?" (couverture). Compromis entre les deux ; le F1 les combine.

Q5. C'est quoi TF-IDF, concrètement ? Fréquence d'un mot dans l'avis × rareté du mot dans l'ensemble du corpus. Les mots fréquents partout (peu utiles) sont dévalués, les mots rares et discriminants valorisés.

Q6. Pourquoi Naive Bayes est "naïf" ? Il suppose que les mots sont **indépendants** entre eux (faux en pratique), mais ça marche étonnamment bien et c'est très rapide en grande dimension.

Q7. Comment évitez-vous le surapprentissage (overfitting) ? Validation croisée 5-fold, séparation train/test, élagage des arbres, régularisation L2 (régression logistique), early stopping (boosting).

Q8. Qu'est-ce que la validation croisée apporte ? Une estimation plus fiable de la performance (moyenne sur 5 découpages) et un réglage des hyperparamètres sans toucher au test.

Q9. Comment réglez-vous les hyperparamètres ? GridSearchCV : on teste une grille de combinaisons, on garde celle qui maximise le macro-F1 en validation croisée, puis on réentraîne sur tout le train.

Q10. Qu'est-ce que SMOTE et pourquoi pas juste dupliquer ? SMOTE crée des exemples minoritaires **synthétiques** par interpolation entre voisins (pas de simples copies) → meilleure généralisation que la duplication.

Q11. Pourquoi SMOTE seulement sur le train ? Sinon on créerait/évaluerait sur des données synthétiques → fuite et score gonflé. On l'insère dans une `Pipeline imblearn` qui ne s'active qu'à l'entraînement.

Q12. PCA ou TruncatedSVD ? Différence ? TruncatedSVD = "PCA pour matrices creuses" : il ne centre pas les données, donc ne densifie pas la matrice TF-IDF. Même but (réduire les dimensions), adapté au texte.

Q13. Pré-élagage vs post-élagage ? Pré = limiter la croissance pendant la construction (profondeur, feuilles min). Post = laisser pousser puis couper (cost-complexity `ccp_alpha`). On fait les deux.

Q14. Qu'est-ce que l'early stopping ? Dans le boosting, arrêter d'ajouter des arbres dès que la validation ne progresse plus (88/500 chez nous) → moins de calcul, moins de surapprentissage.

Q15. Qu'est-ce que la régularisation L2 ? Une pénalité sur la taille des poids du modèle → frontière plus simple, moins de surapprentissage. Réglée par `C` (petit `C` = forte régularisation).

Q16. Pourquoi AdaBoost est-il si mauvais ici ? Ses classifieurs de base sont des "souches" (1 seule décision) ; sur ~4 600 mots creux, une souche ne voit qu'un mot → trop faible.

Q17. Pourquoi les embeddings Word2Vec (pré-entraîné ou interne) déçoivent-ils ? On **moyenne** les vecteurs de mots d'un avis ; sur un texte court, cette moyenne dilue les mots-clés forts ("crash", "love"). Les comptages TF-IDF les préservent. Le Word2Vec pré-entraîné (Google News) n'est pas meilleur que l'interne ici.

Q17 bis. Vous avez utilisé GloVe ? Word2Vec ? On utilise **Word2Vec** (le mot exact du sujet), en deux versions : **pré-entraîné** sur Google News (le « pre-trained Word2Vec » demandé) et entraîné sur nos avis. Pas de GloVe — on reste strictement dans le périmètre du sujet.

Q18. Et BERT ? Le code est fourni (`BertVectorizer`) mais nécessite de télécharger les poids (HuggingFace), indisponible dans notre environnement → désactivé proprement. Un script prêt à l'emploi (`scripts/run_bert_experiment.py`) l'ajoute à la comparaison sur une machine avec accès Internet (ex. Google Colab : `pip install transformers torch` puis lancer le script). Le sujet dit « Try: ... BERT » — c'est une liste d'options, pas une obligation, et on en couvre déjà 4.

Q19. D'où viennent les labels ? Sont-ils fiables ? Des notes 1-5★ (1-2=nég, 3=neutre, 4-5=pos). C'est un proxy standard mais imparfait, surtout pour le 3★ "neutre" ambigu.

Q20. Comment garantissez-vous la reproductibilité ? Une graine unique (`RANDOM_STATE = 42`) partout, et le dataset traité est mis en cache → relancer `python main.py` redonne exactement les mêmes chiffres.

Q21. Que feriez-vous pour améliorer le neutre ? Plus de données neutres, class-weighting, seuils ajustés, ou fine-tuner BERT.

Q22. Pourquoi avoir dédoublé ? Pour éviter qu'un même texte soit en train et en test (fuite). On passe de 10 000 à 5 923 avis uniques.

10. Glossaire express

Terme	Définition courte
Supervisé	apprentissage avec des labels connus
Feature	une variable d'entrée (ici, un mot/colonne du vecteur)
Vectorisation	transformer du texte en vecteurs de nombres
Sparse / dense	beaucoup de zéros / peu de zéros
Stop-words	mots fréquents peu informatifs, retirés
Lemmatisation	ramener un mot à sa forme de base

Terme	Définition courte
Hyperparamètre	réglage fixé avant l'entraînement
Validation croisée	évaluer en tournant sur k découpages
Overfitting	le modèle mémorise le train, généralise mal
Régularisation	pénalité anti-overfitting
SMOTE	génère des exemples minoritaires synthétiques
Macro-F1	moyenne simple des F1 par classe (toutes égales)
Matrice de confusion	tableau vraies classes × classes prédites

11. Antisèche finale (à relire 5 min avant)

- **Tâche** : sentiment d'avis Facebook → négatif / neutre / positif (supervisé).
- **Données** : 5 923 avis uniques ; **64 % pos, 31 % nég, 4.5 % neutre**.
- **Pipeline** : nettoyage → vectorisation → (équilibrage) → (réduction) → modèle réglé par **CV 5-fold + GridSearchCV** → évaluation.
- **Meilleur** : **TF-IDF + Naive Bayes, macro-F1 0.559**, accuracy 0.839.
- **Métriques** : précision = fiabilité ; rappel = couverture ; F1 = combine les deux ; **macro-F1** = les 3 classes à égalité (notre métrique de référence).
- **Le clou** : baseline **accuracy 0.84 mais rappel neutre 0.000** → **SMOTE** le remonte à 0.19, le sous-échantillonnage à **0.47**. Accuracy ≠ vérité sur données déséquilibrées.
- **Réglages montrés** : élagage (1 857 → 69 nœuds, ↑ test), régularisation L2, early stopping (88/500), réduction SVD (300 dims ≈ 99 % de la perf).